

■■ Let's Build a Map Quiz with JavaScript!

A beginner-friendly coding lesson for young learners in India

Subject	Programming — JavaScript Fundamentals
Level	Beginner (Ages 10 – 14)
Duration	60 – 90 minutes (1 session)
Goal	Build an interactive India map quiz using variables, functions, if-else, and click events

■ Learning Objectives

By the end of this lesson, students will be able to:

- Explain what JavaScript is and how it makes webpages interactive
- Create and use **variables** to store information
- Define and call a **function** to reuse code
- Write an **if-else** statement to make decisions
- Respond to a **click event** from a user
- Assemble all four concepts into a working map quiz

■ What You Need

- A computer or laptop with any web browser (Chrome recommended)
- A text editor (Notepad, VS Code, or an online editor like CodePen)
- The starter India map HTML file provided by the teacher
- Curiosity and a love of learning! ■

Section 1 — What is JavaScript? ■■

Before we start coding, let us understand **why** we need JavaScript.

■ HTML — The Shape

HTML gives every webpage its structure — headings, paragraphs, buttons, and images. Think of it as the outline of a Rangoli pattern drawn on the floor.

■ CSS — The Colours

CSS adds colours, fonts, and layout to the HTML structure. It fills the Rangoli with beautiful hues.

■ JavaScript — The Diya that Lights It Up!

JavaScript (JS) makes pages **respond** to the user. When you click a button, type in a box, or hover over an image, JS runs your code and creates the magic on screen.

Without JS, a webpage is like a printed painting — pretty but still. **With JS, it comes alive!**

■ Real-world examples of JavaScript at work:

- The search suggestions that appear as you type on Google
- The score counter in an online quiz
- Swipe-able image carousels on shopping websites
- Live bus / train tracking maps

Section 2 — Our Three Building Blocks ■

2A Variables — The Dabba for Data ■

What is a Variable?

A variable is like a **tiffin dabba** (lunchbox) — it stores a value and you give it a label so you can find it later.

In JavaScript we create a variable with the keyword **let**:

```
let correctState = 'Rajasthan';

// Now the variable correctState holds the text 'Rajasthan'
// We can use it anywhere in our code by typing its name
```

Try it yourself!

Change the value to your favourite state:

```
let correctState = 'Tamil Nadu';
let correctState = 'Maharashtra';
let correctState = 'Kerala';
```

2B Functions — School Experiment Steps ■

What is a Function?

A function bundles a set of instructions so we can **reuse** them. Just like a science experiment has numbered steps that you follow every time, a function groups those steps under one name.

We **define** a function once and **call** it whenever we need it.

```
// Define the function
function checkAnswer(clicked) {
  // Our quiz-checking steps go here
}

// Call the function (run it)
checkAnswer('Rajasthan');
```

2C If-Else — The Cricket Umpire ■

Making Decisions in Code

An if-else statement works like a cricket umpire — it looks at what happened and gives a verdict: OUT or NOT OUT.

When the condition is **true**, the first block runs. Otherwise the **else** block runs.

```
if (answer === correct) {  
  showWin(); // ■ OUT – you got it right!  
} else {  
  tryAgain(); // ■ NOT OUT – keep trying!  
}
```

Section 3 — Click Events: The Doorbell ■

Events — When Something Happens

Every time a user **clicks**, **taps**, or **presses a key**, the browser creates an *event*. JavaScript is always listening for these events — like a doorbell that rings when a visitor arrives.

We attach a listener to an element with **onclick** or **addEventListener**:

Rajasthan

```
// When the user clicks the button, JS immediately calls  
// our checkAnswer function with 'Rajasthan' as the argument.
```

How Our Map Quiz Uses Events

Each Indian state on the map is an HTML element. We attach an **onclick** handler to every state. When a student clicks a state, the browser fires a click event, JS runs **checkAnswer()**, and the quiz responds instantly — just like KBC!

Section 4 — The Complete Quiz Code ■

Now let us see how all four concepts combine into the finished quiz. Read each comment (the lines starting with *//*) carefully.

[illegible]

■ Trace the code — Correct answer

1. Student clicks 'Rajasthan'
2. `onClick` fires →
`checkAnswer('Rajasthan')`
3. `clicked === correctState` → `TRUE`
4. score becomes 10
5. Alert: '■ Sahi jawab!'

- ## ■ Trace the code — Correct answer
1. Student clicks 'Rajasthan'
 2. `onClick` fires →
`checkAnswer('Rajasthan')`
 3. `clicked === correctState` → `TRUE`
 4. score becomes 10
 5. Alert: '■ Sahi jawab!'

■ Trace the code — Wrong answer

1. Student clicks 'Kerala'
2. onClick fires → checkAnswer('Kerala')
3. clicked === correctState → FALSE
4. else block runs
5. Alert: '■ Galat!'

- ## ■ Trace the code — Wrong answer
1. Student clicks 'Kerala'
 2. onClick fires → checkAnswer('Kerala')
 3. clicked === correctState → FALSE
 4. else block runs
 5. Alert: '■ Galat!'

Section 5 — Your Turn! Challenges ■

Modify the starter code to complete each challenge below. Each challenge builds on the previous one.

■ Level 1 — Change the Question

Concept used: `correctState`

Replace 'Rajasthan' with any other Indian state. Run the quiz and check it works.

■■ Level 2 — Better Messages

Concept used: alert text

Make the win message include the student's name: `alert('Shabash ' + studentName + '! Correct!');`

■■■ Level 3 — Add a Second Question

Concept used: second variable

Create a second variable `correctCapital = 'Jaipur'` and a new function `checkCapital(clicked)`. Show a question asking 'What is the capital of Rajasthan?'

■■■■ Level 4 — Score Display

Concept used: DOM update

Instead of `alert()`, display the score inside an HTML element:
`document.getElementById('score').innerText = 'Score: ' + score;`

Section 6 — Quick Reference Card ■

Concept	Analogy	Description
Variable	Dabba 🍷	Stores a value with a name. <code>let x = 5;</code>
Function	Experiment 🧪	Groups reusable steps. <code>function doIt() { ... }</code>
If-Else	Umpire 🏏	Makes decisions. <code>if (a===b) { ... } else { ... }</code>
Click Event	Doorbell 🔔	Runs code when user clicks. <code>onclick='fn()'</code>
Alert	Loudspeaker 📢	Shows a popup message. <code>alert('Hello!');</code>
===	Exact match 🎯	Checks if two values are exactly equal.
let	New dabba 🍷	Creates a new variable.

[illegible]

Section 7 — Wrap-Up & What We Learned ■

■ JavaScript lights up webpages

Just like a Diya brings light to a room, JavaScript brings interactivity to an otherwise static webpage. HTML and CSS build the shape; JS animates it.

■ Variables store information

We used 'let correctState' to hold the name of the right state, and 'let score' to keep track of how many points the player has earned.

■ Functions group steps

Our checkAnswer() function bundles all the quiz logic in one place. We can call it from every state button without rewriting the same code again.

■ If-Else makes decisions

Like a cricket umpire, the if-else block decided whether the student picked the right state (OUT — they win points) or the wrong one (NOT OUT — try again!).

■ Click events connect the user to the code

The onclick attribute turned a boring HTML button into an interactive quiz element. Every click triggers checkAnswer() and the magic happens instantly.

■ Exit Ticket — Think & Reflect

Answer these in your own words before leaving the class:

1. What is the difference between HTML, CSS, and JavaScript?
2. Why do we use a variable instead of typing the answer directly in the if-else?
3. Can you think of a website you use daily that must use JavaScript? Why do you think so?
4. What would happen if we removed the else block from our quiz?

Great job today! You built your first interactive quiz. Keep creating! ■